# Práctica 2:

# Introducción a WEKA y ANACONDA (Python)




**Universidad de Valladolid**

**Técnicas de Aprendizaje Automático**

Grado de Ingeniería Informática
Doble Grado con Estadística (INDAT)
Departamento de Informática (ATC, CCIA y LSI)

# Datos Crédito

- Crear un archivo con la tabla de datos utilizada en la presentación de la asignatura (Inducción de árboles de decisión – Diapositiva 15) para ilustrar la inducción de árboles con ID3 (crédito) en los siguientes formatos:
  - CSV
  - ARFF

Nota: el algoritmo ID3 no está cargado en weka por defecto. Para ello hay que recurrir al menú “Herramientas” y acceder al “administrador de paquetes”

# Árbol de Decisión Crédito (WEKA)

- Inducir un árbol de decisión con el algoritmo ID3:
  - Si se desea visualizar gráficamente el árbol, recurrir a J48:
    - collapseTree = False
    - MinNumObj = 1
    - Unprunned = True
- En las opciones de test, elegir las muestras de aprendizaje
- Comprobar si coincide con el resultado de la clase de teoría

# Arbol de decisión Crédito (Python)

- Usando:
  - from sklearn.tree import DecisionTreeClassifier, export_graphviz
  - inducir el árbol de decisión en python

- Como paso previo, hay que asegurarse de que export_graphviz está instalado (buscar en google)

- Se requiere la aplicación “dot”, que se ejecuta como un comando de linux para obtener un fichero gráfico: jpg, png, pdf, etc.

# Transformación de atributos nominales a numéricos en Python

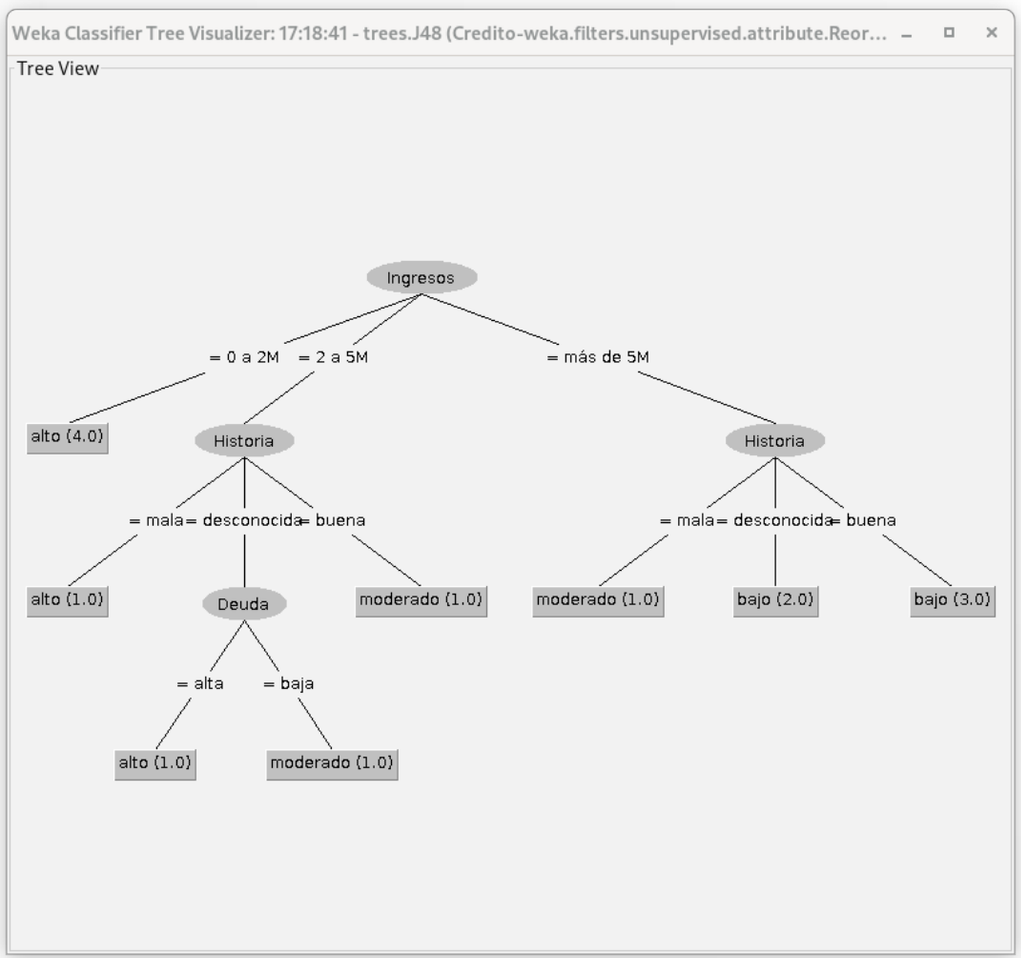
El módulo DecisionTreeClassifier no admite variables nominales, pero sí numéricas. Por este motivo, se ha de llevar a cabo la siguiente transformación:

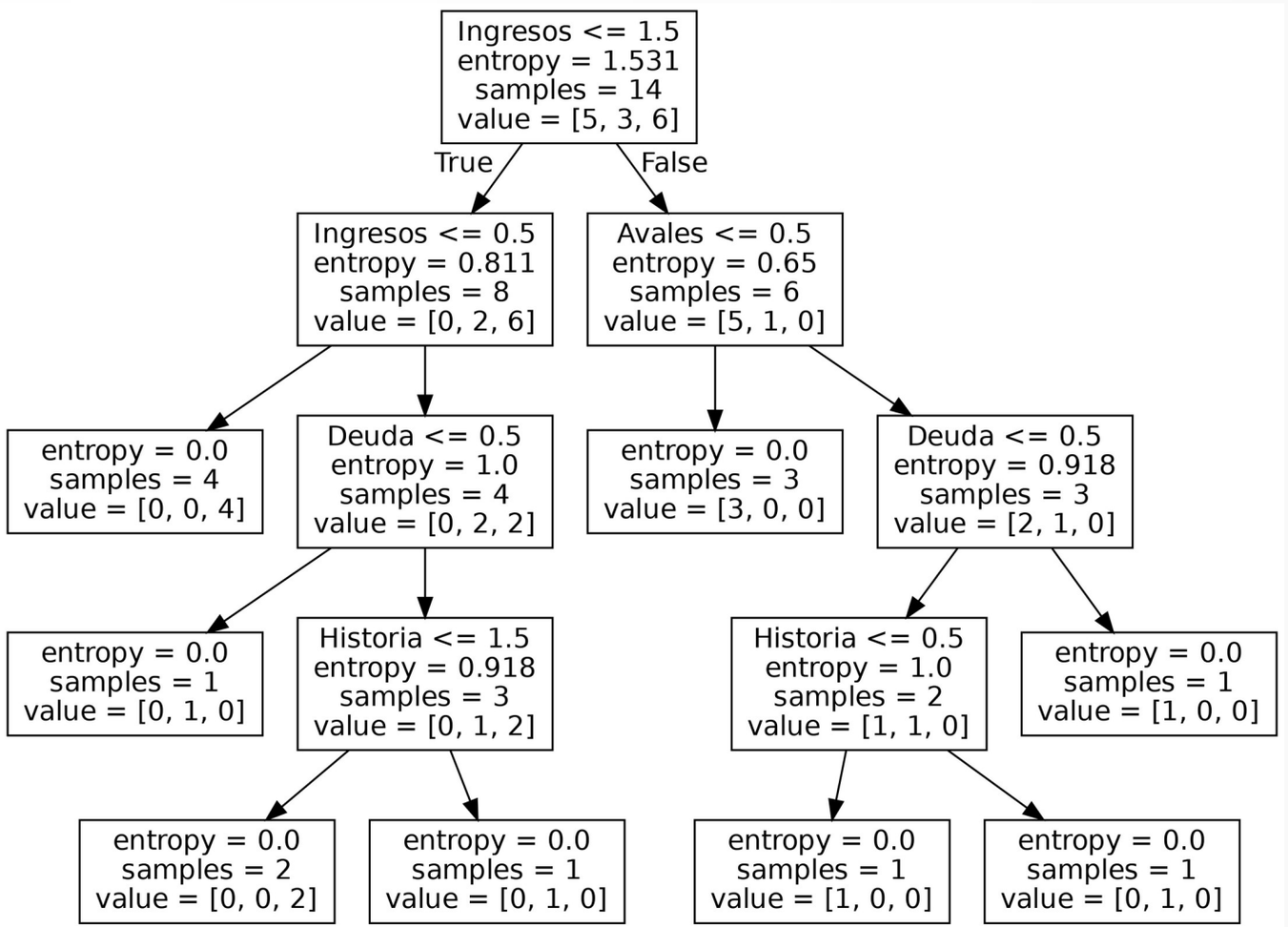
| | Riesgo | Historia | Deuda | Avales | Ingresos |
|---|---|---|---|---|---|
| **0** | 2 | 1 | 1 | 0 | 0 |
| **1** | 2 | 0 | 1 | 0 | 1 |
| **2** | 1 | 0 | 0 | 0 | 1 |
| **3** | 2 | 0 | 0 | 0 | 0 |
| **4** | 0 | 0 | 0 | 0 | 2 |
| **5** | 0 | 0 | 0 | 1 | 2 |
| **6** | 2 | 1 | 0 | 0 | 0 |
| **7** | 1 | 1 | 0 | 1 | 2 |
| **8** | 0 | 2 | 0 | 0 | 2 |
| **9** | 0 | 2 | 1 | 1 | 2 |
| **10** | 2 | 2 | 1 | 0 | 0 |
| **11** | 1 | 2 | 1 | 0 | 1 |
| **12** | 0 | 2 | 1 | 0 | 2 |
| **13** | 2 | 1 | 1 | 0 | 1 |



| | Riesgo | Historia | Deuda | Avales | Ingresos |
|---|---|---|---|---|---|
| **0** | alto | mala | alta | no | 0 a 2M |
| **1** | alto | desconocida | alta | no | 2 a 5M |
| **2** | moderado | desconocida | baja | no | 2 a 5M |
| **3** | alto | desconocida | baja | no | 0 a 2M |
| **4** | bajo | desconocida | baja | no | más de 5M |
| **5** | bajo | desconocida | baja | adecuados | más de 5M |
| **6** | alto | mala | baja | no | 0 a 2M |
| **7** | moderado | mala | baja | adecuados | más de 5M |
| **8** | bajo | buena | baja | no | más de 5M |
| **9** | bajo | buena | alta | adecuados | más de 5M |
| **10** | alto | buena | alta | no | 0 a 2M |
| **11** | moderado | buena | alta | no | 2 a 5M |
| **12** | bajo | buena | alta | no | más de 5M |
| **13** | alto | mala | alta | no | 2 a 5M |

# árboles WEKA (id3) y SKLEARN(CART)



# Ejemplo de función lógica (entrega)

- Tome la siguiente función lógica y obtenga el árbol de decisión (gráfico) correspondiente usando PYTHON.

$$\left(\left(A \wedge B\right) \vee \left(\overline{C} \wedge D\right)\right) xor \left(E \vee \overline{F}\right) \wedge \left(G \vee H \vee I \vee J\right)$$

- Para ello: from sklearn.tree import DecisionTreeClassifier, export_graphviz

- Vuelque la tabla construida a un fichero csv
- Construya el árbol correspondiente con weka, usando id3 y/o j48 con las restricciones de la diapositiva anterior
- Incorpore dicho gráfico al final fichero “jupyter” de esta entrega, así como el obtenido con python.
- Compruebe si son iguales
- Verifique que en ambos casos acierta el 100% de las muestras del experimento